Design Principles

This document records the reasoning behind the structure and conventions of this repository. It exists so that the *why* behind each decision is preserved, not just the *what* — and so that future changes are made with the original intent in mind rather than against it.

---

1. Git as the source of truth

The DBA database historically existed only as a live instance. Changes were made directly against it, with no version history, no review path, and no recovery mechanism other than backups.

Bringing it under source control provides:

- **Version history** — every change is tracked, attributable, and reversible.
- **Review** — changes can be examined before they reach the live database.
- **Recoverability** — the database definition can be rebuilt from the repository.

The repository is authoritative. The live database reflects the repository, not the other way around.

---

2. One object per file

Rather than a single monolithic export script, each object lives in its own file.

The benefit is **meaningful change history**. A diff on `usp_Monitor_CaptureWaitStats.sql` tells the story of that one procedure. A diff against a multi-megabyte combined script tells you almost nothing. Version control is only as useful as the granularity it operates on.

---

3. Deployment order is self-documenting

Top-level folders are numbered to encode dependency order:

- The database and security come first (`00_database/`, `01_security/`).
- Third-party code before the owned objects that call it (`02_third_party/`).
- Types before the tables and procedures that reference them (`03_types/`).
- Tables before the views built on them (`04_tables/`, `05_views/`).
- Views before the procedures that consume them (`06_programmability/`).

Third-party code sits at 02 because owned code depends on it, never the other way round: owned procedures call vendor procedures (for example `usp_Run_Blitz` calls `sp_Blitz`), and two vendor tables, `CommandLog` and `SqlServerVersions`, are used by vendor procedures.

`07_jobs/`, `08_migrations/` and `09_deployment/` follow; `10_metadata/` holds the manifests and generated scripts and is applied last. This makes ordering explicit rather than relying on knowledge held in someone's head. Anyone can read the folder numbers and understand the sequence.

---

4. Separation of owned code from third-party code

Community tooling — Ola Hallengren's maintenance solution, the Brent Ozar First Responder Kit, Adam Machanic's `sp_WhoIsActive` and Erik Darling's utilities — is held in a dedicated, isolated area (`02_third_party/`), one folder per author.

This is the single most important structural decision in the repository. The reasons:

- **Clean diffs.** When an upstream author releases an update, the change is immediately distinguishable from our own work. Mixed together, the two become indistinguishable in history.
- **Clear ownership.** It is always obvious which objects are ours to change and which are not.
- **Protection from accidental edits.** Third-party procedures call one another internally using exact names; an accidental rename or edit can silently break them.

Third-party objects are never renamed or edited. This includes the two vendor tables, `CommandLog` and `SqlServerVersions`, which live in `02_third_party/` under their published names. They remain exactly as published; the only change made on import is `CREATE OR ALTER` in place of `CREATE`, so that redeployment is safe.

---

5. Domain grouping within object types

Within `04_tables/`, `05_views/`, and the procedures area (`06_programmability/procedures/`), objects are grouped into functional domains (waitstats, filestat, cpu, memory, rcsi, and so on).

The reasoning is navigation under pressure. When investigating a CPU problem, going straight to `05_views/cpu/` is faster and less error-prone than scanning one large folder. The structure is designed around how the objects are actually used, not just how SQL Server categorises them.

This sits alongside — not instead of — the type-based numbering. Type governs deployment order; domain governs navigation. Both concerns are served.

---

6. Migrations capture change; object files capture state

The numbered object folders capture current state — what each object looks like now. They do not capture how it got there.

- For views, functions, and procedures this is fine: they are redeployed safely with `CREATE OR ALTER`.
- For tables it is not: a column cannot be added by re-running a CREATE statement.

The `08_migrations/` folder holds forward-only, dated scripts for structural change. This gives table changes a proper home and preserves the history of structural change that current-state files cannot express.

> In short: current-state files answer “what does this object look like?”; migration scripts answer “how did it change, and when?” Both questions matter, and each has its own mechanism.

---

7. Schema usage

All objects deploy to `dbo`. Database schemas are deliberately not used. The repository folder structure carries the logical organisation instead.

Introducing schemas was considered and rejected:

- **Migration cost.** Every reference in Agent jobs, application code, and monitoring queries would need moving from `dbo` to the new schema.
- **Third-party incompatibility.** Ola Hallengren’s procedures call one another using `dbo`-qualified names, as do several Brent Ozar procedures. Relocating them to a schema would mean editing the vendor scripts — directly violating the third-party isolation principle (Section 4).

The folder structure delivers the organisational benefit schemas would have provided, without the disruption.

> Note: Schemas are an organisational and permission-delegation tool, not a security boundary. A db_owner sees everything regardless of schema. The decision to avoid them rests on migration cost and third-party compatibility, not on security.

---

8. Logic belongs in procedures, not in Agent

Scheduled job-step logic historically lived in SQL Server Agent, inside `msdb`, where it is invisible to source control — it cannot be diffed, reviewed, or redeployed consistently.

The principle: the scheduler decides timing; the procedure holds the logic. Each Agent job step is reduced to a thin call (`EXEC dbo.usp_...`), and the logic it invokes lives in a version-controlled, reviewable procedure in this repository.

The analogy is a scheduled task that calls a script, versus one with the logic embedded in the scheduler itself. Logic belongs in the version-controlled artefact, not the scheduler.

---

9. Deployment scripts are generated from a manifest

`10_metadata/object_manifest.csv` records each object's domain, package, dependencies and deploy order. `deploy_all.sql`, the third-party and package scripts, and `apply_extended_properties.sql` are generated from it by `tools/generate_deploy.py`, so ordering is data rather than a hand-maintained script.

The extended properties stamped on each object (`RepositoryGroup`, `PackageName`, `IsThirdParty`) are a copy of that metadata, so the live database can be audited against the repository. The repository remains authoritative (Section 1).